

Callback functions em .Js

#programming

#javascript

Em JavaScript, **callback functions** são funções passadas como argumento para outra função, para serem executadas posteriormente.

Video sobre o tema

[Entenda Callbacks definitivamente \(JavaScript\)](#)

1. Ideia central

Uma callback permite dizer:

“execute isso depois que terminar essa tarefa”

2. Exemplo básico

```
function saudacao(nome) {  
  console.log("Olá " + nome);  
}  
function processar(callback) {  
  callback("Walter");  
}  
processar(saudacao);
```

- `saudacao` é passada como callback
 - `processar` decide quando executá-la
-

3. Como funciona

Quando você faz:

```
processar(saudacao);
```

Você não está executando a função, apenas **passando a referência**.

Depois:

```
callback("Walter");
```

A função é executada.

4. Callbacks assíncronas

Muito usadas com operações que levam tempo:

```
setTimeout(() => {  
  console.log("Executado depois");  
}, 1000);
```

Aqui:

- a função só roda após 1 segundo
 - o restante do código continua rodando normalmente
-

5. Exemplo prático

```
function carregarPagina(callback) {  
  setTimeout(() => {  
    callback();  
  }, 2000);  
}  
  
function pronto() {  
  console.log("Página carregada");  
}  
  
carregarPagina(pronto);
```

6. Por que usar callbacks

- controlar **ordem de execução**
- lidar com **tarefas assíncronas**
- tornar funções mais reutilizáveis

7. Erro comum

```
carregarPagina(pronto());
```

Isso executa a função imediatamente. O correto:

```
carregarPagina(pronto);
```

8. Limitação

Muitas callbacks encadeadas podem gerar código difícil de ler (“callback hell”). Por isso surgiram `Promise` e `async/await`.